

# Miscellaneous

---

## Lecture 01: Vibe Coding

Prof. Dr. Michael Granitzer

# Ziel dieser Vorlesung

Vibe Coding ist keine magische Produktivitätstechnik. Es ist eine Arbeitsweise, in der Menschen mit generativen Modellen Software schneller entwerfen, ändern und prüfen.

Die zentrale Frage ist deshalb nicht: "Kann die AI coden?", sondern: **Unter welchen Arbeitsbedingungen wird daraus verlässliche Entwicklung?**

## Leitfragen

- Warum funktioniert Vibe Coding für manche Teams erstaunlich gut, für andere aber kaum?
- Warum ist **Supervision** wichtiger als Prompt-Eleganz?
- Warum sollte man der AI eher das **Was** als das **Wie** vorgeben?
- Welche Inquiry- und Inspektionsstrategien machen die Arbeit belastbar?
- Welche Art von Tests ist im AI-unterstützten Coding besonders wichtig?

# Warum Vibe Coding funktioniert

---

**Leitfrage:** Welche Bedingungen machen AI-unterstütztes Coding produktiv, statt nur schnell und gefährlich?

# Was mit "Vibe Coding" gemeint ist

## Typische Praxis

- Problem grob beschreiben
- Implementierungsideen erzeugen lassen
- Code direkt iterativ veraendern
- schnell testen, verwerfen, neu ansetzen
- viel ueber Dialog statt ueber Vorabplanung arbeiten

## Typischer Reiz

- niedrige Einstiegshuerde
- hohe Geschwindigkeit bei Boilerplate und Umbauten
- schnelle Exploration unbekannter Libraries oder APIs
- gute Unterstuetzung fuer Prototyping und Glue Code

Vibe Coding ist damit eher ein **interaktiver Arbeitsstil** als eine klar definierte Methode.

# Warum es fuer manche gut funktioniert

| Bedingung                         | Warum hilfreich                                     |
|-----------------------------------|-----------------------------------------------------|
| Guter Problemrahmen               | Die AI bekommt ein klares Zielbild                  |
| Schnelle Feedbackschleifen        | Fehler werden frueh sichtbar                        |
| Gute Tooling-Umgebung             | Diff, Tests, Logs und UI machen Ergebnisse pruefbar |
| Erfahrener Mensch in der Schleife | Schwache Stellen werden erkannt                     |
| Kleine bis mittlere Aufgaben      | Komplexitaet bleibt lokal beherrschbar              |

Vibe Coding skaliert nicht automatisch mit Modellstaerke. Es skaliert oft besser mit **Sichtbarkeit, Kritikfaehigkeit und enger Rueckkopplung.**

# Warum es fuer andere scheitert

| Problem                                     | Typischer Effekt                                    |
|---------------------------------------------|-----------------------------------------------------|
| Ziel unklar                                 | Die AI produziert Aktivitaet statt Fortschritt      |
| Mensch kann Ergebnis nicht beurteilen       | Halluzinationen bleiben unentdeckt                  |
| Zu viel Vertrauen in fluessige Erklaerungen | Scheinplausibilitaet ersetzt Verifikation           |
| Keine gute Inspektionsoberflaeche           | Fehler bleiben unsichtbar                           |
| Kein Test- oder Integrationskontext         | "Sieht gut aus" wird mit "funktioniert" verwechselt |
| Zu grosse Umbauten in einem Schritt         | Lokale Fehler propagieren ins Gesamtsystem          |

Der haeufigste Fehler ist nicht, dass die AI etwas Falsches produziert. Der haeufigste Fehler ist, dass Menschen **es zu spaet merken**.

# Operative Prinzipien

---

**Leitfrage:** Wie arbeitet man mit AI-Coding-Tools so, dass man Kontrolle behält?

# Supervision ist der Engpass

## Die naive Sicht

- Hauptproblem sei Prompting
- bessere Formulierung loese fast alles
- Modellqualitaet bestimme den Erfolg

## Die robustere Sicht

- Hauptproblem ist **Aufsicht**
- jemand muss Zwischenschritte beurteilen
- jemand muss falsche Abzweigungen frueh stoppen
- jemand muss Qualitaetskriterien durchsetzen

Nicht Prompt Engineering ist die knappe Ressource, sondern **kompetente Supervision**.

# Zielorientierung: Sage der AI was, nicht wie

## Oft hilfreicher

- Zielzustand beschreiben
- Randbedingungen nennen
- Invarianten festlegen
- unerwünschtes Verhalten explizit verbieten
- Erfolgskriterien benennen

## Oft weniger hilfreich

- zu frueh die konkrete Implementierung diktieren
- interne Struktur ohne Not festlegen
- Modell auf einen einzigen Loesungspfad einsperren
- die Form der Loesung mit dem Zweck verwechseln

Ein guter Prompt beschreibt haeufig zuerst **das Problem, die Grenzen und den Pruefrahmen** - nicht sofort die interne Mechanik.

# Inquiry-Strategien statt blinder Generierung

| Strategie                           | Nutzen                                |
|-------------------------------------|---------------------------------------|
| Nach Annahmen fragen                | Versteckte Unsicherheit wird sichtbar |
| Alternativen vergleichen lassen     | Loesungsraum wird explizit            |
| Zwischenschritte begruenden lassen  | Denkfehler fallen frueher auf         |
| Risiken und Failure Modes abfragen  | Verifikation wird gezielter           |
| Diffs statt Vollausgabe bevorzugen  | Aenderungen bleiben lesbar            |
| Vor dem Edit erst Analyse verlangen | Lokales Verstaendnis steigt           |

Gute Zusammenarbeit mit AI ist oft weniger "schreib mir Code" und mehr **"zeige mir, wie du das Problem gerade modellierst"**.

# Inspection UI entscheidet ueber Vertrauen

Was man sehen koennen sollte:

- Diff statt nur Endzustand
- Tests und Fehlermeldungen
- relevante Logs
- betroffene Dateien und Stellen
- Browser- oder API-Verhalten
- Build- und Laufzeitstatus

Wenn die Benutzeroberflaeche keine gute Inspektion erlaubt, wird Vertrauen schnell zu **Blindflug**.

Gerade deshalb funktionieren AI-Workflows oft besser in Umgebungen mit Terminal, Git-Diff, Testausgabe und DevTools als in reinen Chatfenstern.

# Produktive Regel: Immer kritisch bleiben

*Gute AI-Unterstützung reduziert Arbeit. Sie ersetzt nicht die Pflicht zum Zweifel.*

## Praktische Konsequenzen

- nie Erklärungen mit Korrektheit verwechseln
- nie "klingt sinnvoll" als Abnahmekriterium verwenden
- immer nach Gegenbeispielen suchen
- immer an Daten, Laufzeit und Integration denken
- immer prüfen, was sich durch eine Änderung implizit mitverändert

# Testen und Grenzen

---

**Leitfrage:** Welche Art von Verifikation passt besonders gut zu AI-unterstütztem Coding?

# Testing: weniger nur Unit, mehr Integration

## Warum Integrationstests wichtiger werden

- AI aendert oft mehrere Schichten gleichzeitig
- Fehler entstehen haeufig an Schnittstellen
- Konfigurations- und Datenflussprobleme sind oft zentral
- lokale Korrektheit garantiert noch kein Systemverhalten

## Typische Pruefpunkte

- API-Endpunkte mit realistischen Requests
- Datenbank- und Persistenzpfade
- Frontend-Backend-Interaktion
- Auth-, Rechte- und Session-Pfade
- Build-, Start- und Deploybarkeit

# Synthetic Data Testing als Arbeitsmittel

| Nutzen                          | Beispiel                                           |
|---------------------------------|----------------------------------------------------|
| Seltene Faelle gezielt erzeugen | Grenzwerte, Nullwerte, leere Antworten             |
| Datenschutz wahren              | keine produktiven Personendaten noetig             |
| Testabdeckung verbreitern       | viele Eingabevarianten systematisch pruefen        |
| Failure Modes provozieren       | kaputte Formate, schlechte Reihenfolgen, Konflikte |

Synthetische Daten sind nicht nur Ersatz fuer echte Daten. Sie sind ein Werkzeug, um **gezielt schwierige Faelle sichtbar zu machen.**

# Wo Vibe Coding besonders riskant bleibt

| Bereich                    | Warum riskant                                   |
|----------------------------|-------------------------------------------------|
| Sicherheitskritische Logik | kleine Fehler haben grosse Wirkung              |
| Verteilte Systeme          | Fehler zeigen sich oft nur emergent             |
| Legacy-Systeme             | implizites Wissen fehlt im Prompt               |
| Regulierte Domänen         | Nachvollziehbarkeit und Compliance sind zentral |
| Grosse Refactorings        | globale Seiteneffekte schwer ueberschaubar      |

Je teurer Fehler werden und je spaeter sie sichtbar sind, desto weniger darf man sich auf "schnelle Plausibilitaet" verlassen.

# Arbeitsmodell: Vibe Coding unter Aufsicht

```
Zielbild und Constraints klaeren
->
AI fuer Analyse, Varianten und Entwurf nutzen
->
kleine Aenderung mit guter Sichtbarkeit erzeugen
->
Diff, Logs, UI und Verhalten inspizieren
->
Integration und Synthetic Data testen
->
erst dann uebernehmen oder verwerfen
```

Vibe Coding wird dann belastbar, wenn es nicht als automatisches Coden verstanden wird, sondern als **schnelle Hypothesenerzeugung unter starker menschlicher Aufsicht**.

# Wrap-Up

- Vibe Coding funktioniert vor allem dort gut, wo **Supervision, Zielklarheit und schnelle Rueckmeldung** vorhanden sind.
- Der Mensch sollte der AI eher **das Was als das Wie** vorgeben: Ziele, Constraints, Nicht-Ziele und Erfolgskriterien.
- Inquiry-Strategien und gute Inspection UI sind entscheidend, weil sie Annahmen, Risiken und Fehler sichtbar machen.
- Kritisches Arbeiten bleibt Pflicht: fluessige Erklaerungen sind kein Beweis.
- Fuer Verifikation sind **Integrationstests und Synthetic Data Testing** besonders wichtig.